WEEK 3 GROUP PRACTICE ASSIGNMENT
Smart Cold-Chain Delivery Monitoring Device

1. Objective

Modify the Week 3 practical code to create a virtual IoT monitoring device for a refrigerated delivery box.

- Generate simulated IoT sensor measurements.
- Validate sensor values and determine operating status.
- Organize measurements using nested Python dictionaries.
- Serialize structured data into JSON.
- Compare normal and compact JSON payload sizes.
- Generate repeated IoT messages and summarize warning conditions.

2. Scenario

A pharmaceutical company uses refrigerated delivery boxes to transport temperature-sensitive products. Each box contains an IoT monitoring device that measures temperature, humidity, and battery level. The device prepares structured data before the measurements are transmitted to a monitoring system.

Measurement	Normal operating range
Temperature	2–8 °C
Humidity	40–70 %
Battery level	30–100 %

3. Instructions

- Use your Week 3 practical program as the starting point.
- Modify and extend the existing code according to the tasks below.
- Do not copy code from another group. Group members should be able to explain the main parts of the submitted program.
- Test the program carefully and make sure the output is readable.

4. Assignment Tasks

Task 1 -- Modify the Virtual Sensors

Modify the sensor-generation section so the virtual device produces the following ranges:

Measurement	Generation range
Temperature	0–12 °C
Humidity	20–85 %
Battery	10–100 %

- Display all three measurements with appropriate units.

Task 2 -- Add Device Information

- Define a unique device ID for the cold-chain monitoring box, for example using the naming pattern coldbox_XX.
- Add the current timestamp to every IoT record.

Task 3 -- Validate the Sensor Measurements

Check whether each generated value is physically valid using the ranges below:

Measurement	Valid range
Temperature	−20 to 50 °C
Humidity	0–100 %
Battery	0–100 %

- Assign a quality result such as VALID or INVALID for each measurement.
- Test your validation by temporarily using at least one invalid value, then restore the virtual sensor generation.

Task 4 -- Determine Operating Status

- Temperature: below 2 °C → TOO_COLD; 2–8 °C → NORMAL; above 8 °C → TOO_WARM.
- Humidity: 30–70 % → NORMAL; otherwise → HUMIDITY_WARNING.
- Battery: below 20 % → LOW_BATTERY; 20 % or higher → NORMAL.
- Write the required conditional logic yourself using if / elif / else statements.

Task 5 -- Create a Structured IoT Record

- Combine the device information and measurements into one Python dictionary.
- Use nested dictionaries to organize the record into the following groups: device_id, timestamp, measurements, quality, and status.
- The measurements group must contain temperature, humidity, and battery.
- The quality and status groups must contain results for the relevant measurements.

Task 6 -- Access Data from the Nested Dictionary

- Display at least three individual values by accessing them from the nested dictionary.
- Include at least one measurement value and at least one status value.

Task 7 -- Serialize the Record to JSON

- Convert the Python dictionary into a JSON string.
- Display both a normal JSON representation and an indented, human-readable JSON representation.

Task 8 -- Check the Data Types

- Display the Python type of the structured record before serialization.
- Display the Python type of the JSON payload after serialization.
- Use the results to explain the difference between a Python dictionary and a JSON string in your report.

Task 9 -- Measure the JSON Payload Size

- Encode the JSON string using UTF-8 and determine its size in bytes.
- Display the measured payload size clearly in the program output.

Task 10 -- Create Compact JSON

- Create a compact JSON representation by removing unnecessary spacing.
- Measure the compact JSON payload size.
- Calculate and display the percentage size reduction compared with the normal JSON representation.

Task 11 -- Generate Repeated IoT Messages

- Extend the program so the device generates 100 IoT records.
- For every record, generate new measurements, validate them, determine status, create the structured record, serialize it, and display the compact JSON payload.
- Use a short delay between consecutive records to simulate periodic device operation.
- Number the records clearly from Reading 1 to Reading 100.

Task 12 -- Count Warning Conditions

- Create counters for temperature warnings, humidity warnings, and low-battery warnings.
- Increase the appropriate counter whenever a warning condition occurs.
- After all 100 records have been generated, display a final summary showing the total number of readings and the number of each warning type.

5. Required Program Flow

1. Generate measurements
2. Add device ID and timestamp
3. Validate measurements
4. Determine operating status
5. Create nested dictionary
6. Serialize to JSON
7. Measure normal JSON size
8. Create compact JSON
9. Compare payload sizes
10. Repeat for 10 records
11. Count warnings
12. Display final summary

6. Group Discussion Questions

1. Why is a nested dictionary more suitable than storing all measurements as separate variables?
2. What is the difference between data validation and operating status?
3. What is the difference between a Python dictionary and JSON?
4. Why is serialization useful when IoT data must be stored or transmitted?

5. Why is compact JSON smaller even though the sensor measurements are unchanged?
6. Why might reducing payload size be useful for an IoT device?

Answer each question briefly (approximately 2–4 sentences).